

SRS AI Schema Builder

AI Schema Builder

Software Requirements Specification (SRS)

Version 1.0 | 2025

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the AI Schema Builder – a web application that assists backend developers in automatically generating, visualizing, and managing database table schemas and their relational references using the Anthropic Claude API.

1.2 Scope

The AI Schema Builder enables development teams to:

Define projects and add tables within those projects

Generate detailed database schemas (columns, types, constraints, indexes) using Claude AI

Automatically detect and visualize table relationships and foreign key references

Export schemas as SQL DDL, JSON, or shareable documentation

Securely manage Anthropic API keys per user or organization

1.3 Definitions & Acronyms

Term

Definition

Schema

The structure definition of a database table including columns, types, and constraints

DDL

Data Definition Language – SQL statements that define database structure

FK

Foreign Key – a column that creates a link between two tables

ERD

Entity Relationship Diagram – a visual map of table relationships

Claude API

Anthropic's AI model API used to generate schema suggestions

SRS

Software Requirements Specification

1.4 References

Google Gemini API Documentation – <https://ai.google.dev/gemini-api/docs>

IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications

2. Overall Description

2.1 Product Perspective

AI Schema Builder is a standalone web application built on a React frontend and Node.js/Express backend. It integrates with the Google Gemini API (gemini-2.5-flash model) to provide AI-powered schema generation. User data (projects, tables, schemas) is stored in a PostgreSQL database. The application is designed for individual developers and small-to-medium engineering teams.

2.2 Product Functions

The primary functions of the system are:

Project Management – create, update, archive, and delete projects

Table Registry – define tables belonging to a project with descriptions

AI Schema Generation – use Gemini to generate column definitions, types, indexes, and constraints

Reference Detection – automatically suggest and manage FK relationships between tables

ERD Visualization – interactive visual diagram of the schema and relationships

Schema Export – export as SQL DDL, JSON, or Markdown documentation

API Key Management – per-user encrypted storage of Google Gemini API keys

2.3 User Classes

User Class

Description

Privileges

Developer

Backend engineer defining schemas

Full CRUD on own projects

Team Admin

Lead engineer managing team projects

View & edit all team projects

Viewer

Stakeholder reviewing schemas

Read-only access

2.4 Assumptions and Dependencies

Users have a free Google Gemini API key from Google AI Studio (aistudio.google.com) – no billing required

Application is deployed to a cloud environment with HTTPS

Target database is relational (PostgreSQL, MySQL, SQLite compatible DDL)

3. Functional Requirements

3.1 User Authentication

FR-01: User Registration

The system shall allow users to register with email and password. Passwords must be hashed using bcrypt with a minimum of 12 rounds.

FR-02: Login & Session

The system shall issue a JWT token upon successful login. Tokens expire after 24 hours. Refresh tokens shall be supported.

3.2 Project Management

FR-03: Create Project

A user shall be able to create a named project with an optional description and target RDBMS type (PostgreSQL, MySQL, SQLite).

FR-04: Project Listing

The system shall display all projects owned by or shared with the authenticated user, with last-modified timestamps.

3.3 Table Management

FR-05: Add Tables to Project

Users shall be able to add one or more named tables to a project, with an optional plain-English description of each table's purpose.

FR-06: Edit Table Metadata

Users shall be able to update the table name and description at any time.

3.4 AI Schema Generation

FR-07: Schema Generation via Gemini

When a user requests schema generation for a table, the system shall call the Gemini API with the project context (project name, existing tables, RDBMS type) and the table's name and description. The response shall include suggested columns, data types, nullable flags, default values, and indexes.

FR-08: Schema Review & Edit

Generated schemas shall be presented in an editable grid. Users can add, remove, or modify columns before saving.

FR-09: Regeneration

Users shall be able to trigger schema regeneration at any time. The previous schema shall be preserved as a version snapshot.

3.5 Reference & Relationship Detection

FR-10: Automatic Reference Suggestion

After schema generation, Gemini shall analyze all table schemas in the project and suggest foreign key relationships. Suggestions include the source table/column and the referenced table/column.

FR-11: Accept / Reject References

Users shall be able to accept or reject each suggested FK relationship. Accepted references are persisted and reflected in the ERD.

3.6 ERD Visualization

FR-12: Interactive ERD

The system shall render an interactive Entity Relationship Diagram using all accepted table schemas and FK references. The diagram shall support zoom, pan, and click-to-inspect.

FR-13: ERD Export

Users shall be able to export the ERD as a PNG or SVG image.

3.7 Schema Export

FR-14: SQL DDL Export

The system shall generate valid CREATE TABLE SQL statements for the selected RDBMS, including PRIMARY KEY, NOT NULL, DEFAULT, UNIQUE, INDEX, and FOREIGN KEY constraints.

FR-15: JSON Export

Schemas shall be exportable as structured JSON for use in ORM configuration tools.

4. Non-Functional Requirements

ID

Category

Requirement

NFR-01

Performance

Schema generation response time < 10 seconds for 95th percentile of requests

NFR-02

Security

Gemini API keys must be encrypted at rest (AES-256)

NFR-03

Security

All communications must use TLS 1.2+

NFR-04

Scalability

System must support 100 concurrent users without degradation

NFR-05

Availability

99.5% uptime SLA for production environment

NFR-06

Usability

Core schema generation workflow completable in under 3 minutes

NFR-07

Data Integrity

All schema mutations shall be journaled and versioned

NFR-08

Portability

Application shall be deployable to AWS, GCP, or Azure

5. System Architecture Overview

5.1 Component Overview

Layer

Technology

Responsibility

Frontend

React + TypeScript + TailwindCSS

UI, ERD visualization (React Flow)

Backend API

Node.js + Express

REST API, business logic, Gemini integration

AI Layer

Google Gemini API (gemini-2.5-flash)

Schema & reference generation

Database

PostgreSQL

User data, projects, schemas, versions

Auth

JWT + bcrypt

Authentication and authorization

Infra

Docker + Kubernetes / AWS ECS

Container orchestration and deployment

5.2 Data Flow

1. User creates a project and adds table names with descriptions.

2. Frontend sends a POST `/api/schemas/generate` request to the backend with project context.

3. Backend constructs a prompt with project context and sends a request to the Gemini API.

4. Gemini returns structured JSON with column definitions, types, and suggested FK references.

5. Backend persists the schema and returns the result to the frontend.

6. Frontend renders the editable schema grid and ERD diagram.

6. API Design

6.1 Core Endpoints

Method

Endpoint

Description

POST

`/api/auth/register`

Register a new user

POST

`/api/auth/login`

Login and receive JWT

GET

`/api/projects`

List all user projects

POST

/api/projects

Create a new project

GET

/api/projects/:id/tables

Get all tables for a project

POST

/api/projects/:id/tables

Add a table to a project

POST

/api/schemas/generate

Generate schema for a table via Gemini

POST

/api/schemas/references

Detect FK references across all tables

GET

/api/schemas/:tableId/export/sql

Export schema as SQL DDL

GET

/api/schemas/:tableId/export/json

Export schema as JSON

GET

/api/projects/:id/erd

Get ERD data (nodes + edges)

7. Constraints & Limitations

Gemini free tier limits apply (250 req/day, 10 RPM) – system must handle 429 responses with exponential backoff

Gemini API keys are free to obtain from aistudio.google.com – no credit card required

Maximum of 200 tables per project to ensure ERD readability

Schema generation is advisory – users are responsible for final schema decisions

The application does not execute SQL against live databases – it is a design-time tool only

End of Document